

1. Define a Tree

Definition:

A **Tree** is a non-linear data structure that represents a hierarchical relationship between elements. It consists of **nodes** connected by **edges**. The topmost node is called the **root**, and the nodes below it are called **children**. Nodes with no children are called **leaves**.

Key Properties:

- 1. **Hierarchical Structure:** Each node (except the root) has exactly one parent.
- 2. **No Cycles:** There are no closed loops in a tree.
- 3. **Connected:** There is exactly one path between any two nodes.

Sample Tree:

Copy



- **Root:** A
- **Children of A:** B, C
- **Children of B:** D, E
- **Children of C:** F
- **Leaves:** D, E, F

Step-by-Step Explanation:

1. Start with the **root node** (A).
 2. Add child nodes (B and C) to the root.
 3. Add child nodes (D and E) to node B.
 4. Add child node (F) to node C.
 5. Nodes D, E, and F have no children, so they are **leaves**.
-

2. Terminology Related to Trees (10 Marks)

Key Terms:

1. **Root:** The topmost node (A in the sample tree).
2. **Parent:** A node with one or more children (e.g., A is the parent of B and C).
3. **Child:** A node with a parent (e.g., B and C are children of A).
4. **Leaf:** A node with no children (e.g., D, E, F).
5. **Sibling:** Nodes with the same parent (e.g., B and C are siblings).
6. **Degree:** Number of children a node has (e.g., degree of A is 2).
7. **Depth:** Number of edges from the root to a node (e.g., depth of D is 2).
8. **Height:** Number of edges on the longest path from a node to a leaf (e.g., height of the tree is 2).
9. **Subtree:** A tree formed by a node and its descendants (e.g., subtree rooted at B includes D and E).
10. **Ancestor:** Nodes on the path from the root to a node (e.g., ancestors of D are A and B).
11. **Descendant:** Nodes reachable from a node by moving down the tree (e.g., descendants of A are B, C, D, E, F).

Sample Tree:

```
  A
 / \
B   C
 / \  \
D  E  F
```

- **Root:** A
- **Parent of D:** B
- **Children of B:** D, E
- **Leaves:** D, E, F
- **Siblings:** B and C
- **Degree of A:** 2
- **Depth of F:** 2
- **Height of the tree:** 2
- **Subtree rooted at B:** D and E
- **Ancestors of E:** A and B
- **Descendants of A:** B, C, D, E, F

3. Define Binary Tree

Definition:

A **Binary Tree** is a tree data structure where each node has at most **two children**, referred to as the **left child** and the **right child**.

Key Properties:

1. Each node has 0, 1, or 2 children.
2. The left child is smaller than the parent, and the right child is larger (in a Binary Search Tree).

Sample Binary Tree:

```
    10
   /  \
  5    15
 / \   \
3  7   20
```

- **Root:** 10
- **Left child of 10:** 5
- **Right child of 10:** 15
- **Left child of 5:** 3
- **Right child of 5:** 7
- **Right child of 15:** 20

Step-by-Step Explanation:

1. Start with the root node (10).
2. Add left child (5) and right child (15) to the root.
3. Add left child (3) and right child (7) to node 5.
4. Add right child (20) to node 15.
5. Nodes 3, 7, and 20 have no children, so they are **leaves**.

4. Linear Representation and Linked List Representation of a Binary Tree

Linear Representation (Array Representation):

- The root is stored at index 0.
- For a node at index i :
 - Left child: $2i + 1$
 - Right child: $2i + 2$
 - Parent: $(i-1)/2$ (integer division)

Sample Binary Tree:

```
    10
   / \
  5   15
 / \   \
3  7  20
```

- **Array Representation:** [10, 5, 15, 3, 7, null, 20]
 - Index 0: 10 (root)
 - Index 1: 5 (left child of 10)
 - Index 2: 15 (right child of 10)
 - Index 3: 3 (left child of 5)
 - Index 4: 7 (right child of 5)
 - Index 5: null (left child of 15 is empty)

- Index 6: 20 (right child of 15)

Linked List Representation:

- Each node is represented as a structure with three fields:
 - Data
 - Pointer to the left child
 - Pointer to the right child

Sample Binary Tree:

```
    10
   /  \
  5    15
 / \   \
3  7   20
```

- **Linked List Representation:**
 - Node 10: Left → 5, Right → 15
 - Node 5: Left → 3, Right → 7
 - Node 15: Left → null, Right → 20
 - Nodes 3, 7, and 20: Left → null, Right → null

5. Define Binary Search Tree

Definition:

A **Binary Search Tree (BST)** is a binary tree with the following properties:

1. The left subtree of a node contains only nodes with values **less than** the node's value.
2. The right subtree of a node contains only nodes with values **greater than** the node's value.
3. Both the left and right subtrees must also be binary search trees.

Sample BST:

```
    10
   /  \
  5    15
 / \   \
3  7  20
```

- **In-order Traversal:** 3, 5, 7, 10, 15, 20 (sorted order).

6. Insertion, Deletion, Search, and Traversal Operations on a Binary Search Tree

Insertion:

1. Start from the root.
2. Compare the value to be inserted with the current node.
3. If the value is less, move to the left child; if greater, move to the right child.
4. Repeat until an empty spot is found, then insert the new node.

Example: Insert 12 into the BST:

10

/ \

5 15

/\ \

3 7 20

- Compare 12 with 10 → Move to right (15).
- Compare 12 with 15 → Move to left (null).
- Insert 12 as the left child of 15.

Deletion:

1. **Case 1:** Node has no children (leaf node) – Simply remove it.
2. **Case 2:** Node has one child – Replace the node with its child.
3. **Case 3:** Node has two children – Find the **in-order successor** (smallest in the right subtree), replace the node with it, and delete the successor.

Example: Delete 15 from the BST:

- Find in-order successor of 15 → 20.
- Replace 15 with 20.
- Delete 20.

Search:

1. Start from the root.
2. Compare the search value with the current node.
3. If the value matches, return the node.
4. If the value is less, move to the left child; if greater, move to the right child.
5. Repeat until the value is found or the tree is exhausted.

Example: Search for 7 in the BST:

- Compare 7 with 10 → Move to left (5).
- Compare 7 with 5 → Move to right (7).
- Found 7.

Traversal:

1. **In-order Traversal:** Left → Root → Right (3, 5, 7, 10, 15, 20).
2. **Pre-order Traversal:** Root → Left → Right (10, 5, 3, 7, 15, 20).
3. **Post-order Traversal:** Left → Right → Root (3, 7, 5, 20, 15, 10).
4. **Level-order Traversal:** 10, 5, 15, 3, 7, 20.

7. Applications of Trees

1. **File Systems:** Represent directories and files hierarchically.
2. **Database Indexing:** BSTs and B-trees are used for efficient data retrieval.
3. **Expression Trees:** Represent arithmetic expressions for evaluation.
4. **Decision Trees:** Used in machine learning for classification.
5. **Huffman Coding:** Used in data compression.
6. **Game Trees:** Represent possible moves in games like chess.
7. **Organization Charts:** Represent hierarchical structures in organizations.